



Disciplina: Teste de Software

Testes funcionais – parte 1

Técnicas de teste

- Critérios para especificação dos casos de teste
- Teste funcional (caixa-preta)
- Teste estrutural (caixa-branca)
- Teste baseado em defeitos

Testes funcionais

- Teste caixa-preta
- Utilização de entradas e saídas esperadas
- Comparação das saídas correntes com as saídas esperadas
- Dependência da especificação do produto de software – documentação
- Dependência da experiência do testador

Testes funcionais

- Passos básicos para aplicar um critério de teste funcional
 - A documentação (especificações e requisitos) é analisada
 - Entradas válidas são escolhidas para determinar se o produto em teste comporta-se corretamente
 - Entradas inválidas são escolhidas para verificar se essas são detectadas e manipuladas adequadamente

Testes funcionais

- Passos básicos para aplicar um critério de teste funcional (continuação)
 - Os casos de testes são construídos (saídas são determinadas para cada entrada)
 - O conjunto de teste é executado e as saídas obtidas são comparadas com as saídas esperadas
 - Um relatório é gerado para avaliar o resultado dos testes

Testes funcionais

- Alguns critérios de testes funcionais
 - Particionamento em Classes de Equivalência: divide o domínio de entrada (e de saída) de um programa em classes de equivalência, a partir das quais derivam-se os casos de teste
 - Análise do Valor Limite: permite derivar casos de teste nos limites (fronteiras) das entradas
 - Teste Funcional Sistemático: os casos de testes são derivados a partir dos limites das classes

Testes funcionais

- Alguns critérios de testes funcionais
 - Grafo de Causa-Efeito: verifica o efeito combinado de dados de entrada e de saída. Causas (condições de entrada) e efeitos (ações e saídas) são combinados em um grafo

Testes funcionais

- Particionamento em Classes de Equivalência
 - Critério utilizado para reduzir o número de casos de teste, procurando garantir uma boa cobertura do código do produto em teste
 - Empregado intuitivamente pelos programadores mesmo sem conhecer o critério

Testes funcionais

- Particionamento em Classes de Equivalência
 - Exemplo: parte de um Sistema de Recursos Humanos que determina contratações conforme a idade dos candidatos
 - Como derivar os casos de teste?

Testes funcionais

- Particionamento em Classes de Equivalência
 - Exemplo: parte de um Sistema de Recursos Humanos

Idade (Anos)	Contratação
De 0 a 15	Não contratar
De 16 a 17	Contratar como aprendiz
De 18 a 70	Contratar como empregado
De 71 a 120	Não contratar

Testes funcionais

- Particionamento em Classes de Equivalência
 - Uma forma de implementação de parte do software
 - `if (idade == 0) printf("Não contratar");`
 - `if (idade == 1) printf("Não contratar");`
 - ...
 - `if (idade == 16) printf("Aprendiz");`
 - `if (idade == 17) printf("Aprendiz");`
 - ...
 - `if (idade == 20) printf("Empregado");`
 - ...
 - `if (idade == 60) printf("Empregado");`
 - `if (idade == 61) printf("Empregado");`
 - ...

Testes funcionais

- Particionamento em Classes de Equivalência
 - Uma forma de testar o software para a implementação do slide anterior seria executá-lo com valores de idade entre 0 e 120, considerando as entradas válidas
 - Caso haja tempo suficiente, esse é o melhor teste a ser realizado

Testes funcionais

- Particionamento em Classes de Equivalência
 - Outra implementação para o mesmo problema
 - `if (idade >= 0 && idade < 16) printf("Não contratar");`
 - `if (idade >= 16 && idade < 18) printf("Aprendiz");`
 - `if (idade >= 18 && idade < 71) printf("Empregado");`
 - `if (idade >= 71 && idade < 120) printf("Não contratar");`

Testes funcionais

- Particionamento em Classes de Equivalência
 - Uma forma de testar o software para a implementação do slide anterior seria pensar em um valor por classe
 - Não é necessário testar para todos os valores de uma classe
 - Apenas uma parte dos valores é testada

Testes funcionais

- Particionamento em Classes de Equivalência
 - Qualquer valor dentro do intervalo tem a mesma relevância, ou seja, qualquer valor escolhido é adequado
 - O mesmo se aplica para os demais intervalos de dados
 - Cada intervalo é chamado de Classe de Equivalência
 - Qualquer valor no intervalo de uma classe é considerado equivalente em termos de teste
 - Premissa: se um caso de teste de uma classe de equivalência revela uma falha, qualquer caso de teste da mesma classe também revelaria

Testes funcionais

- Particionamento em Classes de Equivalência
 - Esse critério assume que existe uma indicação precisa das classes de equivalência
 - `if (idade >= 0 && idade <= 15) printf("Não contratar");`
 - `if (idade > 15 && idade < 18) printf("Aprendiz");`
 - `if (idade >= 18 && nome == "NomedeumaPessoa")
printf("Empregado");`
 - `if (idade >= 71 && idade < 120) printf("Não contratar");`

Testes funcionais

- Particionamento em Classes de Equivalência
 - No exemplo, o critério de teste reduziu o número de casos de teste com entradas válidas de 121 para 4
 - Casos de teste inválidos devem ser levados em consideração

Testes funcionais

- Passos para aplicação do critério Particionamento em Classes de Equivalência
 - Análise da documentação
 - Identificar as classes de equivalência (requisitos de teste do critério)
 - Condições de entrada
 - Classes válidas e inválidas

Testes funcionais

- Passos para aplicação do critério Particionamento em Classes de Equivalência (continuação)
 - Definir os casos de teste
 - Enumerar as classes de equivalência
 - Criar casos de teste para as classes de equivalência válidas
 - Criar um caso de teste para cada classe de equivalência inválida
 - Entradas inválidas são fontes de defeitos

Testes funcionais

- Passos para aplicação do critério Particionamento em Classes de Equivalência (continuação)
 - Casos de teste adicionais podem ser criados caso haja tempo e dinheiro suficientes
 - Execução dos casos de testes no programa
 - Relatório da execução, indicando os casos que revelaram falhas

Testes funcionais

- Particionamento em Classes de Equivalência
 - Exemplo: programa *String*
 - O programa *String* solicita do usuário um inteiro positivo no intervalo entre 1 e 20, então solicita uma cadeia de caracteres desse comprimento. Em seguida, o programa solicita um caractere e retorna a posição na cadeia em que o caractere foi encontrado pela primeira vez; ou uma mensagem indicando que o caractere não está presente na cadeia. O usuário tem a opção (S para continuar e N para não continuar) de procurar por outro caractere na cadeia, somente um de cada vez, quantas vezes desejar.

Testes funcionais

- Particionamento em Classes de Equivalência
 - Aplicação do critério

Condições de entrada	Classes válidas	Classes inválidas

Testes funcionais

- Particionamento em Classes de Equivalência
 - Identificação das condições de entrada

Condições de entrada	Classes válidas	Classes inválidas
Tamanho (T)		
Cadeia (CC)		
Caractere (C)		
Opção de continuar (O)		

Testes funcionais

- Particionamento em Classes de Equivalência
 - Identificação das classes válidas e inválidas

Condições de entrada	Classes válidas	Classes inválidas
Tamanho (T)	Valores inteiros e $1 \leq T \leq 20$ (1)	$T < 1$ (7) $T > 20$ (8)
Cadeia (CC)	Qualquer sequência de caracteres e tamanho da CC = T (2)	Qualquer sequência de caracteres e tamanho da CC $\neq T$ (9)
Caractere (C)	Qualquer caractere, $C = 1$ e C pertence a CC (3) Qualquer caractere, $C = 1$ e C não pertence a CC (4)	Qualquer caractere e tamanho de C $\neq 1$ (10)
Opção de continuar (O)	$O = \{S\}$ (5) $O = \{N\}$ (6)	$O \neq \{S, N\}$ (11)

Testes funcionais

- Particionamento em Classes de Equivalência
 - Criação dos casos de teste
 - Para as entradas, a primeira é o comprimento da cadeia de caracteres, a segunda é a cadeia (*string*), a terceira é caractere a ser buscado na cadeia, a quarta é a opção (continuar a busca ou parar). A cada continuação um novo caractere para busca deve ser inserido, bem como uma nova opção, gerando outra saída

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 1
 - Descrição: para testar o programa String
 - Entradas: **4**, **"aula"**, **a**, **N**
 - Saída: 1
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 2
 - Descrição: para testar o programa String
 - Entradas: **4**, **"aula"**, **x**, **N**
 - Saída: “Mensagem: não pertence”
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 3
 - Descrição: para testar o programa String
 - Entradas: **4, "aula", a, S, l, N**
 - Saídas: 1, 3
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas e ordem das saídas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 4
 - Descrição: para testar o programa String
 - Entradas: **-2, "aula", u, N**
 - Saída: “Mensagem: comprimento da cadeia é inválido”
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 5
 - Descrição: para testar o programa String
 - Entradas: **4**, **"aul"**, **a**, **N**
 - Saída: “Mensagem: cadeia de caracteres inválida”
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 6
 - Descrição: para testar o programa String
 - Entradas: **4**, **"aula"**, **a**, **X**
 - Saída: “Mensagem: opção de continuação inválida”
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

- Particionamento em Classes de Equivalência
- Caso de teste 7
 - Descrição: para testar o programa String
 - Entradas: **25, "aula", a, N**
 - Saída: “Mensagem: tamanho da cadeia inválido”
 - Pré-condições: -
 - Procedimento de execução: ordem das entradas

Testes funcionais

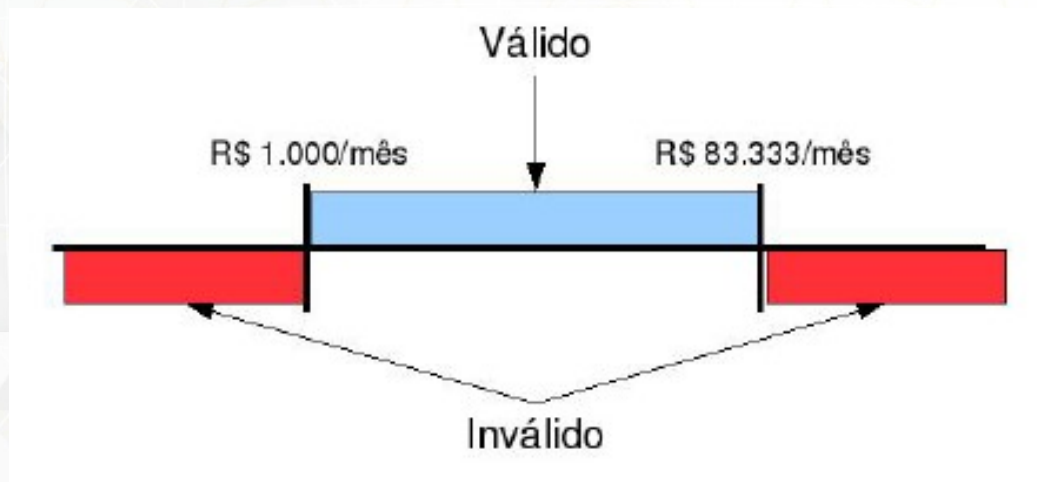
- Particionamento em Classes de Equivalência
 - Exemplo: sistema de hipotecas (Profa. Simone Senger Souza - ICMC/USP)
 - Classes de equivalência para diferentes tipos de dados
 - Classes para Dados Contínuos (renda mensal para hipoteca deve estar entre R\$ 1.000,00 e R\$ 83.333,00)

Testes funcionais

- Particionamento em Classes de Equivalência
 - São definidas duas classes inválidas e uma válida
 - Para a classe válida poderia ser escolhido R\$ 1.342,00 por mês
 - Para as classes inválidas poderiam ser escolhidos R\$ 123,00 por mês e R\$ 90.000,00 por mês

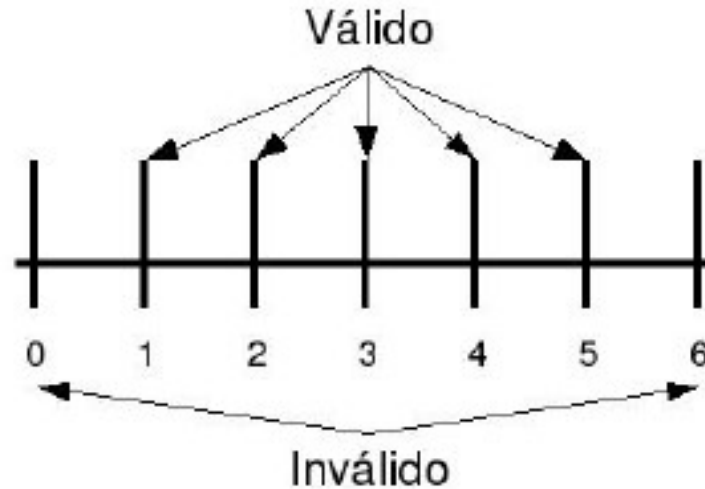
Testes funcionais

- Particionamento em Classes de Equivalência
 - São definidas duas classes inválidas e uma válida



Testes funcionais

- Particionamento em Classes de Equivalência
 - Dados discretos
 - Hipotecas de 1 a 5 casas

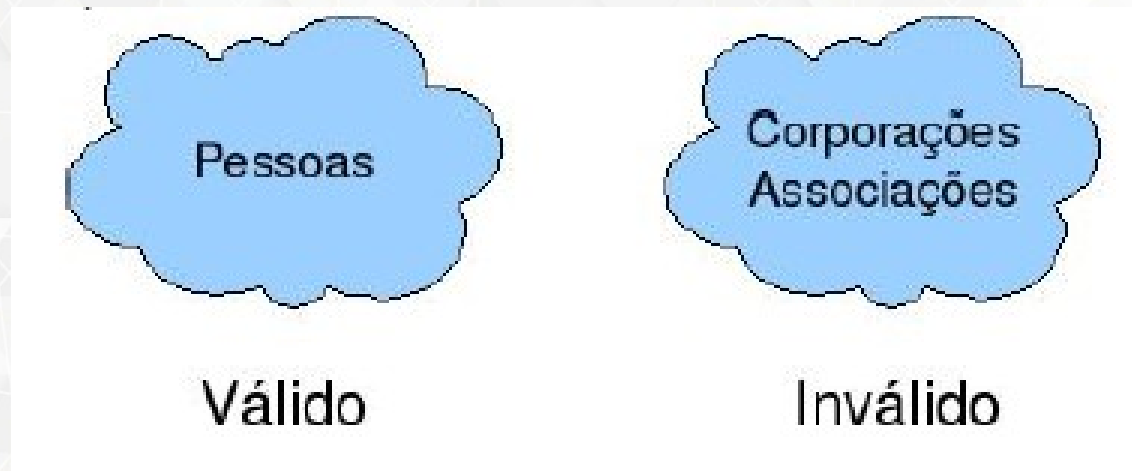


Testes funcionais

- Particionamento em Classes de Equivalência
 - São definidas duas classes inválidas e uma válida
 - Para a classe válida pode-se escolher o valor 2
 - Para as classes inválidas pode-se escolher os valores -2 e 8

Testes funcionais

- Particionamento em Classes de Equivalência
 - Classes para dados simples
 - Somente permissão de hipotecas para pessoas



Testes funcionais

- Particionamento em Classes de Equivalência
 - São definidas uma classe inválida e uma classe válida
 - Para a classe válida pode-se escolher uma determinada pessoa
 - Para a classe inválida pode-se escolher uma empresa ou associação

Testes funcionais

- Particionamento em Classes de Equivalência
 - Classes para dados de múltiplas escolhas
 - Três tipos de hipotecas são permitidas: condomínio, sobrado e casa térrea



Válido



Inválido

Testes funcionais

- Particionamento em Classes de Equivalência
 - Para o intervalo válido pode-se escolher condomínio, sobrado ou casa térrea
 - Escolher somente um ou os três? Depende da criticalidade do programa em teste. Se forem poucos itens vale a pena selecionar um de cada
 - Para a classe inválida pode-se escolher duplex e trailer

Testes funcionais

- Particionamento em Classes de Equivalência
 - Possível solução
 - Criação do menor número possível de casos de teste para cobertura de todas as classes válidas
 - Criação de um caso de teste para cada classe inválida

Testes funcionais

- Particionamento em Classes de Equivalência
 - Possíveis entradas e saídas para criação de casos de teste

Renda	# Casas	Aplicante	Tipo	Resultado
5.000	2	Pessoas	Condomínio	Válido
100	1	Pessoas	Casa Térrea	Inválido
90.000	1	Pessoas	Casa Térrea	Inválido
1.342	0	Pessoas	Condomínio	Inválido
1.342	6	Pessoas	Condomínio	Inválido
1.342	1	Corporação	Sobrado	Inválido
1.342	1	Pessoas	Duplex	Inválido

Testes funcionais

- Particionamento em Classes de Equivalência
 - Benefícios e limitações
 - Reduz significativamente o número de casos de teste comparando com o teste exaustivo
 - Mais adequado para o teste de produtos com domínios de entrada que possam ser divididos em intervalos
 - Assume que os valores dentro da mesma classe são equivalentes - **nem sempre é verdade**

Testes funcionais

- Particionamento em Classes de Equivalência
 - Benefícios e limitações
 - Importante empregar outros critérios de teste
 - Aplicação em todas as fases de teste: unidade, integração e sistema

Testes funcionais

- Análise do Valor Limite
 - Permite criar casos de teste nos limites ou fronteiras de intervalos
 - Pode ser combinada com o critério de Particionamento em Classes de Equivalência

Testes funcionais

- Análise do Valor Limite
 - Passos para aplicação
 - Identificar os intervalos, similar a identificação das classes de equivalência a partir da documentação
 - Identificar os limites de cada classe
 - Criar os casos de testes para os limites
 - Um ponto acima do limite
 - Um ponto do limite
 - Um ponto abaixo do limite

Testes funcionais

- Análise do Valor Limite
 - Passos (Continuação)
 - Casos de teste adicionais podem ser criados dependendo dos recursos disponíveis
 - Observe que acima e abaixo são termos relativos e dependentes do valor dos dados
 - Número inteiros: limite = 16; abaixo = 15; acima = 17
 - Número reais: limite = R\$ 5,00; abaixo = R\$ 4,99; acima = R\$ 5,01

Testes funcionais

- Análise do Valor Limite
 - Exemplo utilizado anteriormente

Idade (Anos)	Contratação
De 0 a 15	Não contratar
De 16 a 17	Contratar como aprendiz
De 18 a 70	Contratar como empregado
De 71 a 120	Não contratar

Testes funcionais

- Análise do Valor Limite
 - Valores a serem considerados:
 - Intervalo entre 0 e 15: $\{-1, 0\}, \{15, 16\}$
 - Intervalo entre 16 e 17: $\{17, 18\}$ (15 e 16 já foram considerados)
 - Intervalo entre 18 e 70: $\{70, 71\}$ (17 e 18 já foram considerados)
 - Intervalo entre 71 e 120: $\{120, 121\}$ (70 e 71 já foram considerados)

Testes funcionais

- Análise do Valor Limite
 - Se tiver recursos, como tempo para teste, outros valores limites a serem considerados: {-1, 0, 1}, {14, 15, 16}, {17, 18}, {19}, {70, 71, 72}, {119, 120, 121}

Idade (Anos)	Contratação
De 0 a 15	Não contratar
De 16 a 17	Contratar como aprendiz
De 18 a 70	Contratar como empregado
De 71 a 120	Não contratar

Testes funcionais

- Análise do Valor Limite
 - Determinar casos de testes utilizando o critério Análise do Valor Limite para o programa *String*

Testes funcionais

- Análise do Valor Limite
 - Benefícios e Limitações
 - Reduz significativamente o número de casos de teste quando comparado com o teste exaustivo
 - Mais adequado para o teste em software com domínio de entrada que pode ser dividido em intervalos
 - Aplicação em todas as fases de teste

Testes funcionais

- Considerações
 - Técnicas funcionais podem ser utilizadas em todas as fases de teste
 - Independente do paradigma de programação utilizado
 - Eficaz para revelar determinadas falhas. Por exemplo: funcionalidade ausente no software
 - Dependente da experiência do testador
 - Muitas combinações das classes podem ser criadas
 - Não considera dependências e sequências nas entradas.

Testes funcionais

- Considerações
 - Dependente de uma boa especificação de requisitos
 - Especificações descritivas e informais podem inviabilizar a utilização do critérios
 - Requisitos imprecisos podem inviabilizar a utilização dos critérios
 - Impossibilidade de garantir que partes essenciais do software sejam executadas

Dúvidas

Testes funcionais

- Exercício
 - Gerar casos de teste utilizando os critérios de Particionamento em Classe de Equivalência e Análise do Valor Limite
 - Software: programa para determinar se um identificador de variáveis de uma linguagem de programação é válido ou inválido. Um identificador válido deve começar com uma letra e conter caracteres alfanuméricos. Além disso, deve conter no mínimo um caractere e no máximo seis caracteres de comprimento